

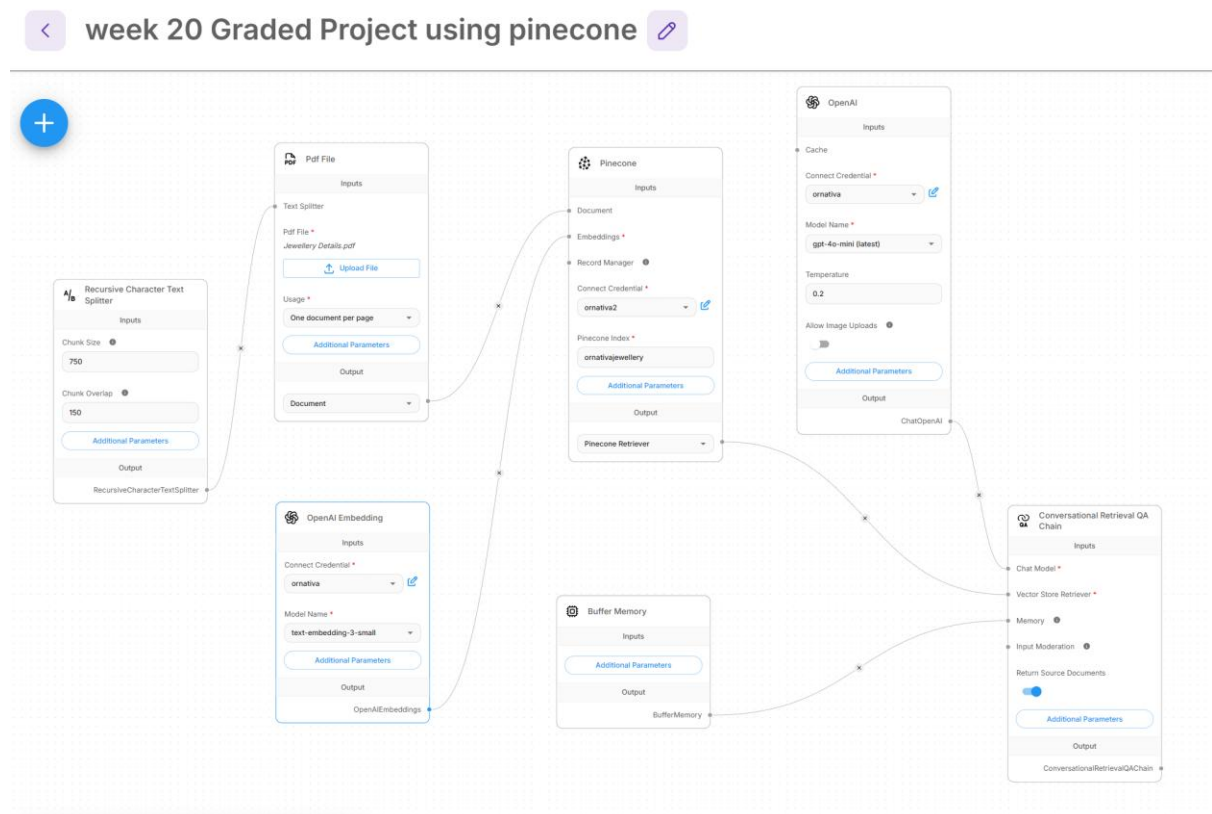
Ornativa's Jewellery Assistant

Overview:

This project builds a **Retrieval-Augmented Generation (RAG) chatbot** for a jewellery catalogue using Flowise. The chatbot is designed to answer user queries strictly based on catalogue data, ensuring accurate and grounded responses.

It supports follow-up questions through memory, maintains a polite and factual tone, and provides alternative product suggestions when items are out of stock, delivering a smooth and context-aware user experience.

Workflow:



Node Choices

- **PDF Loader** – Loads the jewellery catalogue document
- **Recursive Character Text Splitter** – Splits the document into smaller chunks for processing
- **OpenAI Embeddings** – Converts text data into vector embeddings. For this workflow, I used the text-embedding-3-small model.
- **Vector Store (Pinecone)** – Stores embeddings for efficient retrieval

Pinecone was used as the vector database for this project due to its efficient similarity search, which enables fast and accurate retrieval of relevant data based on user queries. It offers a scalable architecture, making it suitable for handling large datasets in real-world applications. As a fully managed service, Pinecone eliminates the need for manual infrastructure management, indexing, and optimization. Additionally, its low-latency performance ensures quick response times, and it

integrates seamlessly with Flowise and embedding models, making it an ideal choice for building RAG-based systems.

- **Conversational Retrieval Chain** – Retrieves relevant chunks based on user queries and applies built-in prompt instructions to control response behavior. It also integrates buffer memory to maintain conversational context across interactions.
- **Chat Model (GPT-4o-mini)** – Generates final responses based strictly on retrieved context and does not use external knowledge.
- **Buffer Memory** – Maintains short-term conversational context across multiple queries. It helps the chatbot handle follow-up questions by remembering previous interactions within the session.

Chunking Approach

- Chunk Size: 750
- Chunk Overlap: 150

This approach ensures:

- Better distribution of data across chunks
- Improved retrieval of relevant information
- Preservation of context between chunks using overlap

During testing, I observed that a small chunk size was causing fragmentation of product data, leading to incomplete context retrieval. To address this, I configured the RecursiveCharacterTextSplitter with a chunk size of 750 and an overlap of 150. This configuration ensures better context preservation and reduces data fragmentation, ensuring that the full product metadata—including stock status—is passed effectively to the LLM. This improves retrieval accuracy and response consistency.

Prompt Design

The chatbot uses a **retrieval-augmented generation (RAG) prompt structure** where the final response is strictly conditioned on retrieved context from the vector database.

The prompt is designed as a **system-level instruction within the Conversational Retrieval Chain**, which enforces the following behavior:

- The **system message ensures grounding**, meaning the model is restricted to answering only from retrieved catalogue context.
- The response generation step is isolated from external knowledge, reducing hallucinations.
- The model is instructed to maintain a **polite, concise, and factual tone** at all times.
- The chatbot explicitly avoids using any information outside the retrieved documents.

In addition, the prompt includes structured fallback and business logic:

- If relevant context is not found, the system returns:
"I am sorry, I do not have information about that in the current catalogue."

- If a product is marked as **out of stock**, the prompt instructs the model to:
 - Clearly inform the user about unavailability
 - Suggest similar or alternative products from the retrieved catalogue

This prompt design ensures controlled, context-aware responses while maintaining reliability and consistency across user queries in the RAG pipeline.

Memory Setup

The chatbot uses **Buffer Memory** within the Conversational Retrieval Chain to maintain short-term conversational context across user interactions.

- The **Buffer Memory node stores recent chat history**, allowing the system to retain context from previous user queries within the same session.
- This enables the chatbot to handle **multi-turn conversations** effectively, especially follow-up questions without requiring the user to repeat context.
- The memory is integrated directly with the retrieval chain, ensuring that both **previous conversation history and newly retrieved context** are considered while generating responses.

Example:

User: Is Emerald Choker available?

Bot: Yes, it is available

User: What is its price?

Bot: 310000

Out-of-Stock Handling

Out-of-stock handling is implemented using **prompt-based logic within the Conversational Retrieval Chain**. The model is instructed to check product availability from retrieved context and clearly inform the user if an item is unavailable.

When a product is out of stock, the chatbot:

- Notifies the user about unavailability
- Suggests similar alternative products from the catalogue

This ensures responses remain accurate, helpful, and fully grounded in the retrieved data.

Example:

“Ruby Solitaire Ring is currently out of stock. You may consider the Classic Diamond Ring, which is available.”

Conclusion

The chatbot successfully demonstrates a complete implementation of a **Retrieval-Augmented Generation (RAG) system** using Flowise.

It is capable of:

- Retrieving relevant information using vector-based similarity search
- Maintaining conversational context through memory integration
- Generating accurate, grounded, and context-aware responses
- Handling out-of-stock scenarios with appropriate fallback and alternatives

Overall, this project showcases an effective end-to-end RAG pipeline for a jewellery catalogue chatbot, combining document retrieval, embeddings, LLM generation, and memory for a coherent user experience.